

Controlo de erros

Exceções

UA.DETI.POO

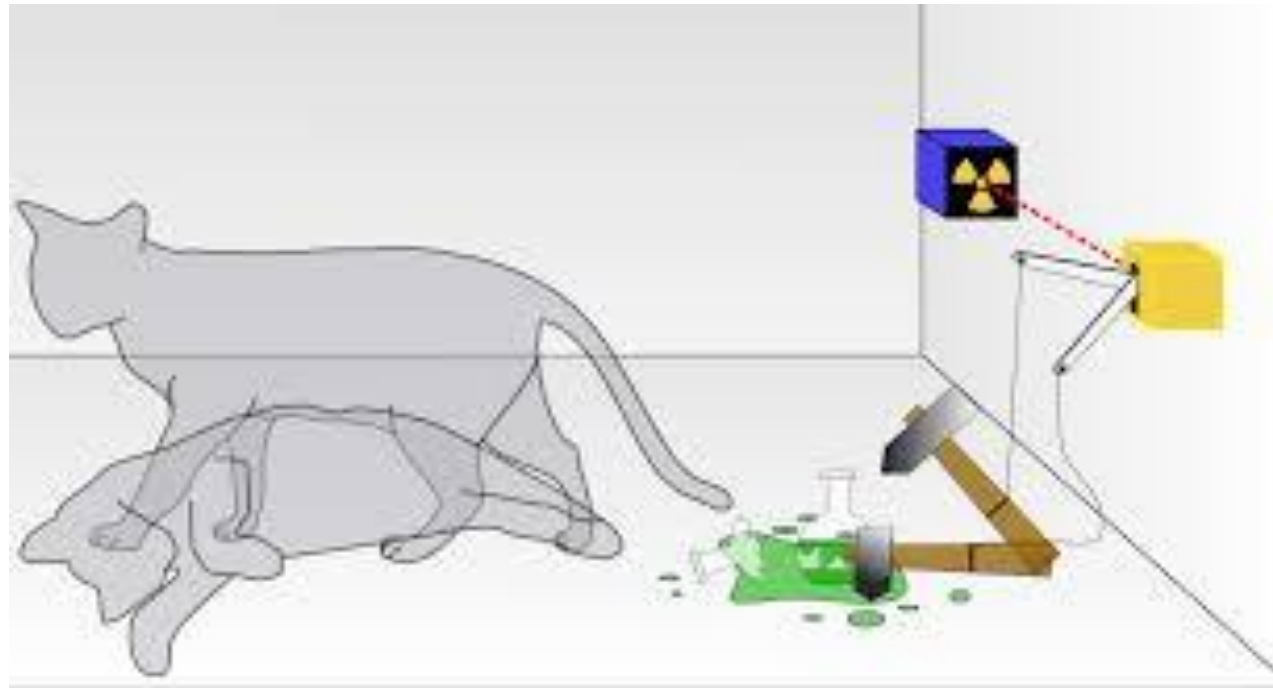
Problema

- ❖ Nem todos os erros são detetados na compilação.
- ❖ Estes são, geralmente, os que são mais menosprezados pelos programadores:
 - Compila sem erros → Funciona!!!
- ❖ Tratamento Clássico:
 - **Se sabemos** à partida que pode surgir uma situação de erro em determinada passagem podemos tratá-la nesse contexto (if...)

```
if ((i = doTheJob()) != -1) {  
    /* tratamento de erro */  
}
```

Exceção

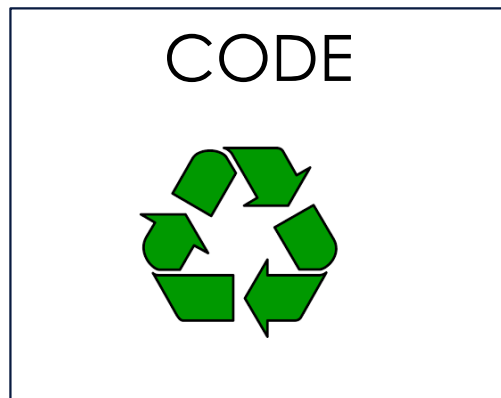
- ❖ Uma exceção é gerada por algo imprevisto que não é possível controlar.



Utilização de Exceções

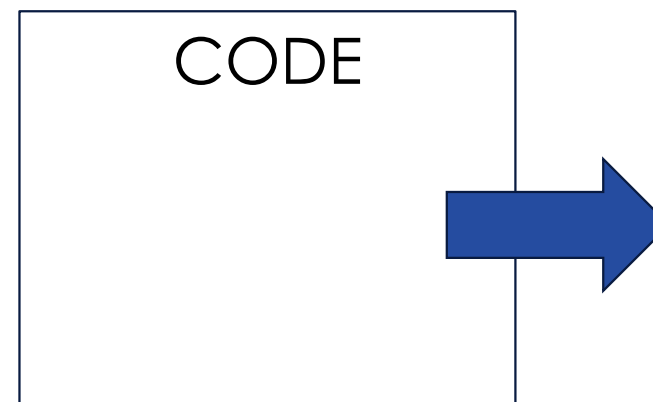
- ❖ Tratamento do erro no contexto local

```
try {  
    /* O que se pretende fazer */  
}  
catch (ExceptionType e) {  
}
```



- ❖ Delegação do erro - gerar um objecto exceção (throw) no qual se delega esse tratamento.

```
if (t == null)  
    throw new NullPointerException();  
    // ou  
    // throw new NullPointerException("can't be null.");
```



shutterstock.com · 146362586

Controlo de Exceções

- ❖ A manipulação de exceções é feita através de um bloco especial: **try .. catch**.

```
try {  
    // Code that might generate exceptions Type1,  
    // Type2 or Type3  
} catch(Type1 id1) {  
    // Handle exceptions of Type1  
} catch(Type2 id2) {  
    // Handle exceptions of Type2  
} catch(Type3 id3) {  
    // Handle exceptions of Type3  
} finally {  
    // Bloco executado independentemente de haver ou não  
    // uma exceção  
}
```



Controlo de Exceções

- ❖ A manipulação de exceções pode ainda ser feita através de um bloco **try-with-resources**.
 - Assegura que os recursos são fechados

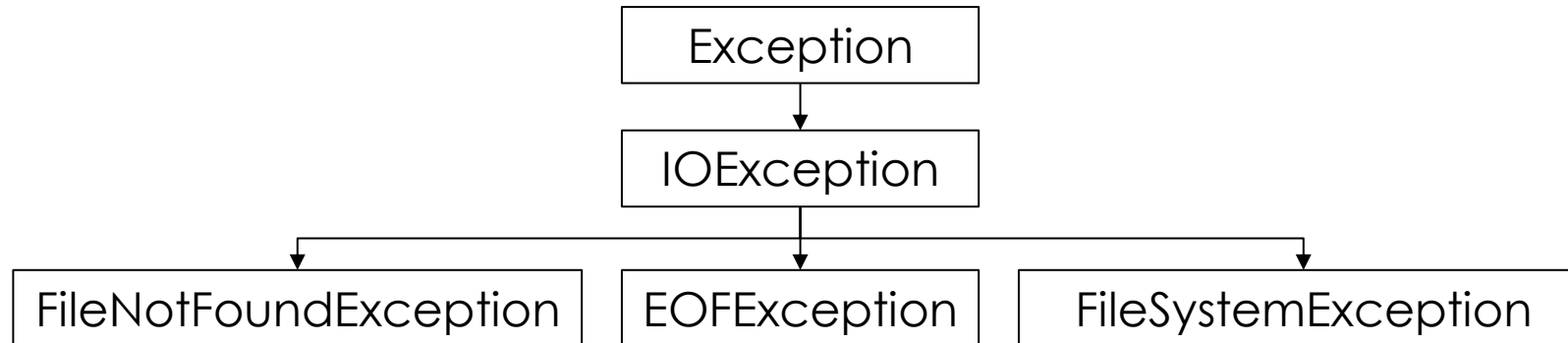


```
try ( resource specification; might
generate exceptions Type1, Type2 or
Type3 )
{
    // code
} catch(Type1 id1) {
    // Handle exceptions of Type1
} catch(Type2 id2) {
    // Handle exceptions of Type2
} catch(Type3 id3) {
    // Handle exceptions of Type3
}
```

Vantagens das Exceções

- ❖ Separação clara entre o código regular e o código de tratamento de erros
- ❖ Propagação dos erros em chamadas sucessivas
- ❖ Agrupamento de erros por tipos

Agrupamento de erros por tipos



```
catch (FileNotFoundException e) {
```

```
    ...  
}
```

Diferenciação

```
catch (IOException e) {
```

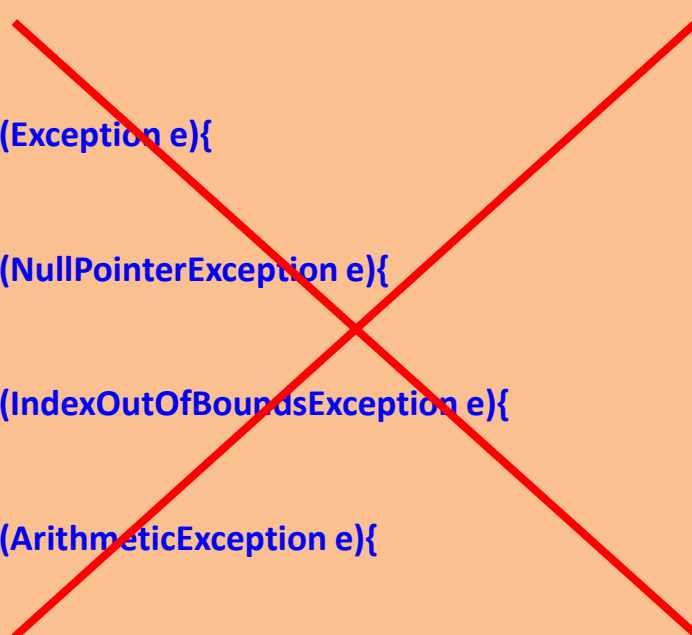
```
    ...  
}
```

Agrupamento

Exceções - Hierarquia de Classes

A ordem dos *catch* é importante

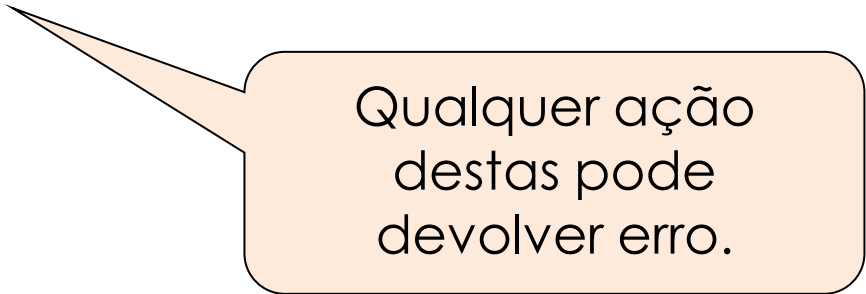
```
try {  
    ...  
}  
catch (Exception e){  
    ...  
}  
catch (NullPointerException e){  
    ...  
}  
catch (IndexOutOfBoundsException e){  
    ...  
}  
catch (ArithmeticException e){  
    ...  
}
```



```
try {  
    ...  
}  
catch (NullPointerException e){  
    ...  
}  
catch (IndexOutOfBoundsException e){  
    ...  
}  
catch (ArithmeticException e){  
    ...  
}  
catch (Exception e){  
    ...  
}
```

Separação de código – exemplo (1)

```
readFile {  
    open the file;  
    determine its size;  
    allocate that much memory;  
    read the file into memory;  
    close the file;  
}
```



Qualquer ação destas pode devolver erro.

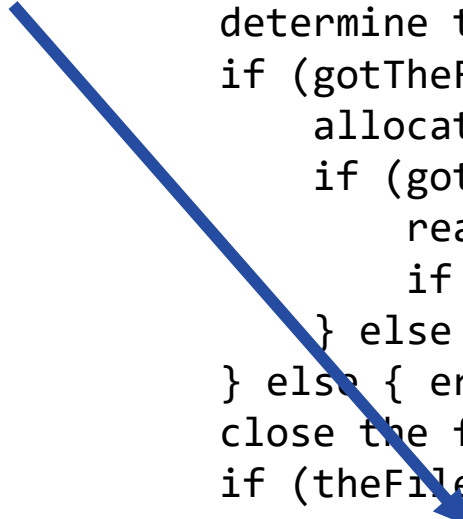
Separação de código – exemplo (2)

Sem Exceções

```
errorCodeType readFile {
    initialize errorCode = 0;
    open the file;
    if (theFileIsOpen) {
        determine the length of the file;
        if (gotTheFileLength) {
            allocate that much memory;
            if (gotEnoughMemory) {
                read the file into memory;
                if (readFailed) { errorCode = -1; }
            } else { errorCode = -2; }
        } else { errorCode = -3; }
        close the file;
        if (theFileDidntClose && errorCode == 0) {
            errorCode = -4;
        } else { errorCode = errorCode and -4; }
    } else { errorCode = -5; }
    return errorCode;
}
```

Separação de código – exemplo (2)

Sem Exceções



```
errorCodeType readFile {
    initialize errorCode = 0;
    open the file;
    if (theFileIsOpen) {
        determine the length of the file;
        if (gotTheFileLength) {
            allocate that much memory;
            if (gotEnoughMemory) {
                read the file into memory;
                if (readFailed) { errorCode = -1; }
            } else { errorCode = -2; }
        } else { errorCode = -3; }
        close the file;
        if (theFileDidntClose && errorCode == 0) {
            errorCode = -4;
        } else { errorCode = errorCode and -4; }
    } else { errorCode = -5; }
    return errorCode;
}
```

Separação de código – exemplo (3)

```
readFile {  
    try {  
        open the file;  
        determine its size;  
        allocate that much memory;  
        read the file into memory;  
        close the file;  
    } catch (fileOpenFailed) {  
        doSomething;  
    } catch (sizeDeterminationFailed) {  
        doSomething;  
    } catch (memoryAllocationFailed) {  
        doSomething;  
    } catch (readFailed) {  
        doSomething;  
    } catch (fileCloseFailed) {  
        doSomething;  
    }  
}
```



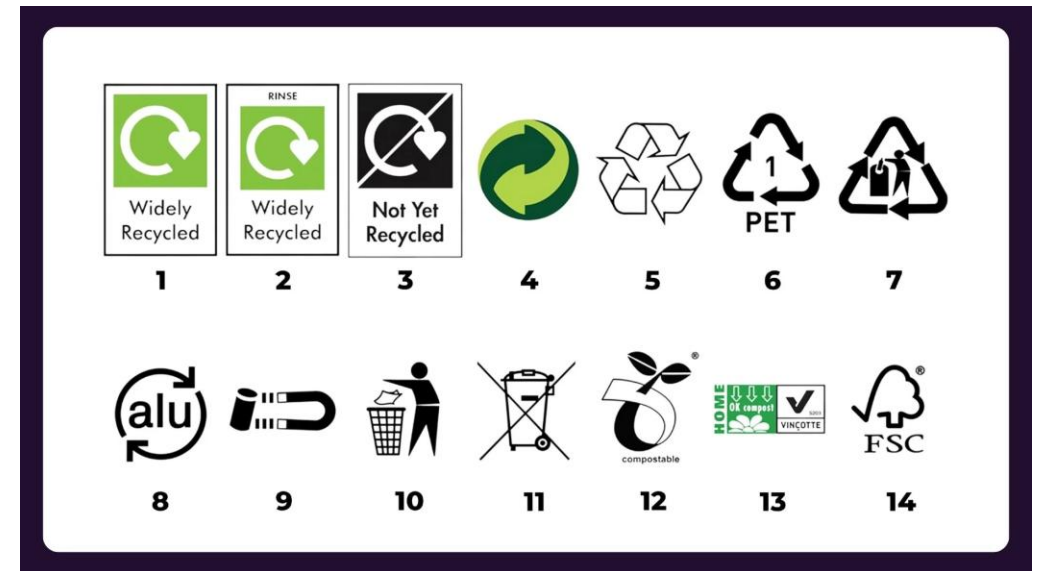
Com Exceções

Declaração throws

- ❖ Quando desenhamos métodos que possam gerar exceções, temos **de assinalá-las explicitamente**

- ❖ Declaração **throws**

```
public void istoPodeDarAsneira()  
    throws TooBigException,  
    TooSmallException,  
    DivByZeroException {  
  
    //...  
  
}
```



Criar Novas Exceções

- ❖ Podemos usar o mecanismo de herança para personalizar algumas exceções

```
class MyException extends Exception {  
    // interface base  
    public MyException() {}  
    public MyException(String msg) {  
        super(msg);  
    }  
    // podemos acrescentar construtores e dados  
}
```

```
// Custom Unchecked Exception
class DivideByZeroException extends RuntimeException {
    public DivideByZeroException(String m) {
        super(m);
    }
}

// Using the Custom Exception
public class Geeks {
    public static void divide(int a, int b) {
        if (b == 0) {
            throw new DivideByZeroException("Division by zero is not allowed.");
        }
        System.out.println("Result: " + (a / b));
    }

    public static void main(String[] args) {
        try {
            divide(10, 0);
        } catch (DivideByZeroException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

<https://www.geeksforgeeks.org/user-defined-custom-exception-in-java/>
<https://www.baeldung.com/java-new-custom-exception>


```
// A Class that represents user-defined exception
class MyException extends Exception {
    public MyException(String m) {
        super(m);
    }
}

// A Class that uses the above MyException
public class setText {
    public static void main(String args[]) {
        try {
            // Throw an object of user-defined exception
            throw new MyException("This is a custom exception");
        }
        catch (MyException ex) {
            System.out.println("Caught"); // Catch and print message
            System.out.println(ex.getMessage());
        }
    }
}
```

<https://www.geeksforgeeks.org/user-defined-custom-exception-in-java/>
<https://www.baeldung.com/java-new-custom-exception>

```
// A Class that represents user-defined exception
class MyException extends Exception {
    public MyException(String m) {
        super(m);
    }
}

// A Class that uses the above MyException
public class setText {
    public static void main(String args[]) {
        try {
            // Throw an object of user-defined exception
            throw new MyException("This is a custom exception");
        }
        catch (MyException ex) {
            System.out.println("Caught"); // Catch and print message
            System.out.println(ex.getMessage());
        }
    }
}
```

<https://www.geeksforgeeks.org/user-defined-custom-exception-in-java/>
<https://www.baeldung.com/java-new-custom-exception>

Propagação dos erros

Propagação dos erros

```
method1 {  
    call method2;  
}  
method2 {  
    call method3;  
}  
method3 {  
    call readFile;  
}
```

Propagação dos erros: no exceptions

```
method1 {  
    errorCodeType error;  
    error = call method2;  
    if (error)    doErrorProcessing;  
    else         proceed;  
}
```

```
errorCodeType method2 {  
    errorCodeType error;  
    error = call method3;  
    if (error)    return error;  
    else         proceed;  
}
```

```
errorCodeType method3 {  
    errorCodeType error;  
    error = call readFile;  
    if (error)    return error;  
    else         proceed;  
}
```

```
method1 {  
    call method2;  
}  
method2 {  
    call method3;  
}  
method3 {  
    call readFile;  
}
```

Propagação dos erros: no exceptions

```
method1 {  
    errorCodeType error;  
    error = call method2;  
    if (error)    doErrorProcessing;  
    else          proceed;  
}
```

```
errorCodeType method2 {  
    errorCodeType error;  
    error = call method3;  
    if (error)    return error;  
    else          proceed;  
}
```

```
errorCodeType method3 {  
    errorCodeType error;  
    error = call readFile;  
    if (error)    return error;  
    else          proceed;  
}
```

```
method1 {  
    call method2;  
}  
method2 {  
    call method3;  
}  
method3 {  
    call readFile;  
}
```

Propagação dos erros: exceptions

```
method1 {
    errorCodeType error;
    error = call method2;
    if (error)    doErrorProcessing;
    else        proceed;
}

errorCodeType method2 {
    errorCodeType error;
    error = call method3;
    if (error)        return error;
    else                proceed;
}

errorCodeType method3 {
    errorCodeType error;
    error = call readFile;
    if (error)        return error;
    else                proceed;
}
```

```
method1 {
    call method2;
}
method2 {
    call method3;
}
method3 {
    call readFile;
}
```

```
method1 {
    try {
        call method2;
    } catch (exception) {
        doErrorProcessing;
    }
}

method2 throws exception {
    call method3;
}

method3 throws exception {
    call readFile;
}
```

Propagação dos erros: exceptions

```
method1 {
    errorCodeType error;
    error = call method2;
    if (error)    doErrorProcessing;
    else          proceed;
}

errorCodeType method2 {
    errorCodeType error;
    error = call method3;
    if (error)    return error;
    else          proceed;
}

errorCodeType method3 {
    errorCodeType error;
    error = call readFile;
    if (error)    return error;
    else          proceed;
}
```

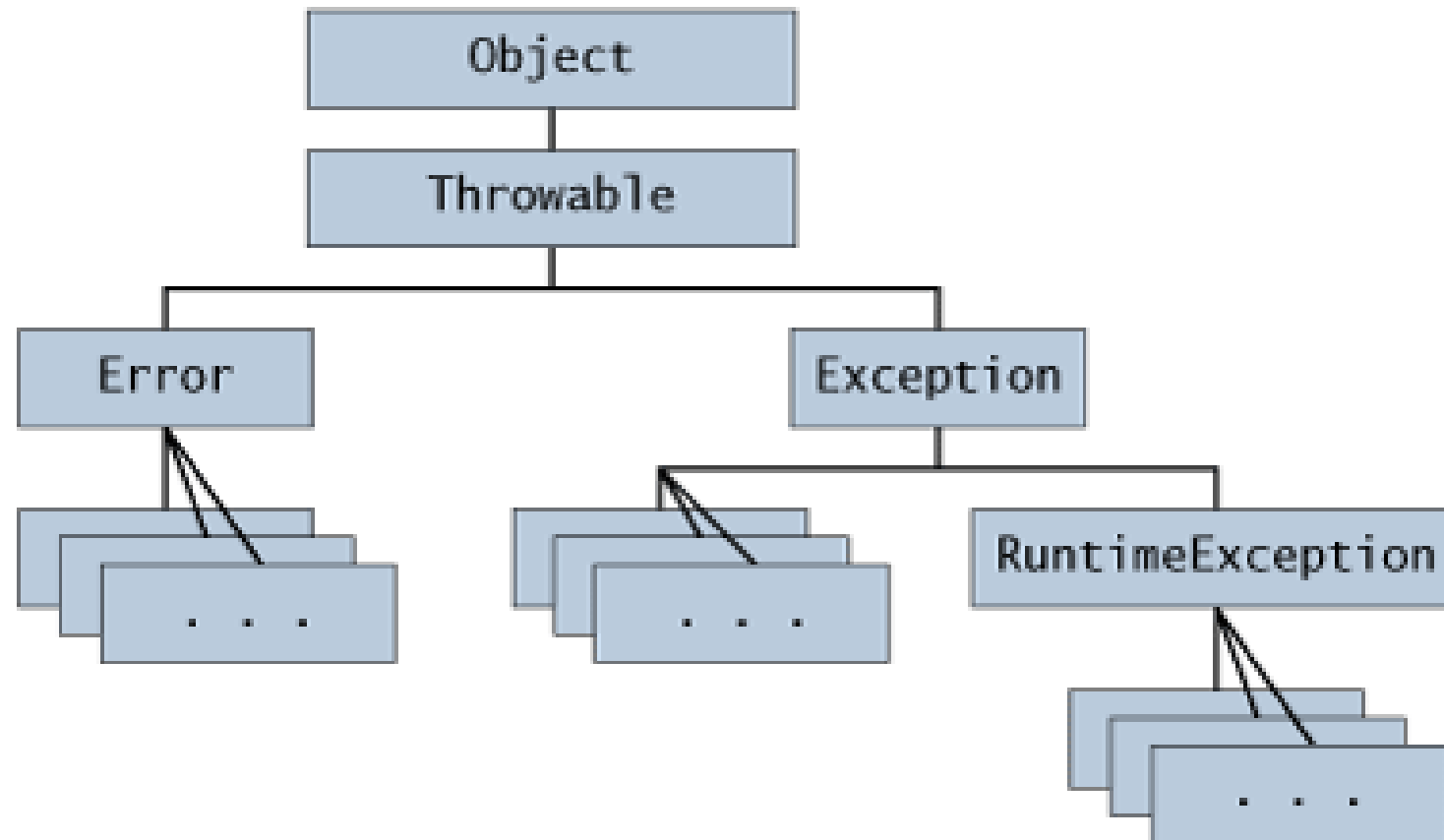
```
method1 {
    call method2;
}
method2 {
    call method3;
}
method3 {
    call readFile;
}
```

```
method1 {
    try {
        call method2;
    } catch (exception) {
        doErrorProcessing;
    }
}

method2 throws exception {
    call method3;
}

method3 throws exception {
    call readFile;
}
```


Tipos de Exceções (e erros)



Tipos de Exceções : checked vs unchecked

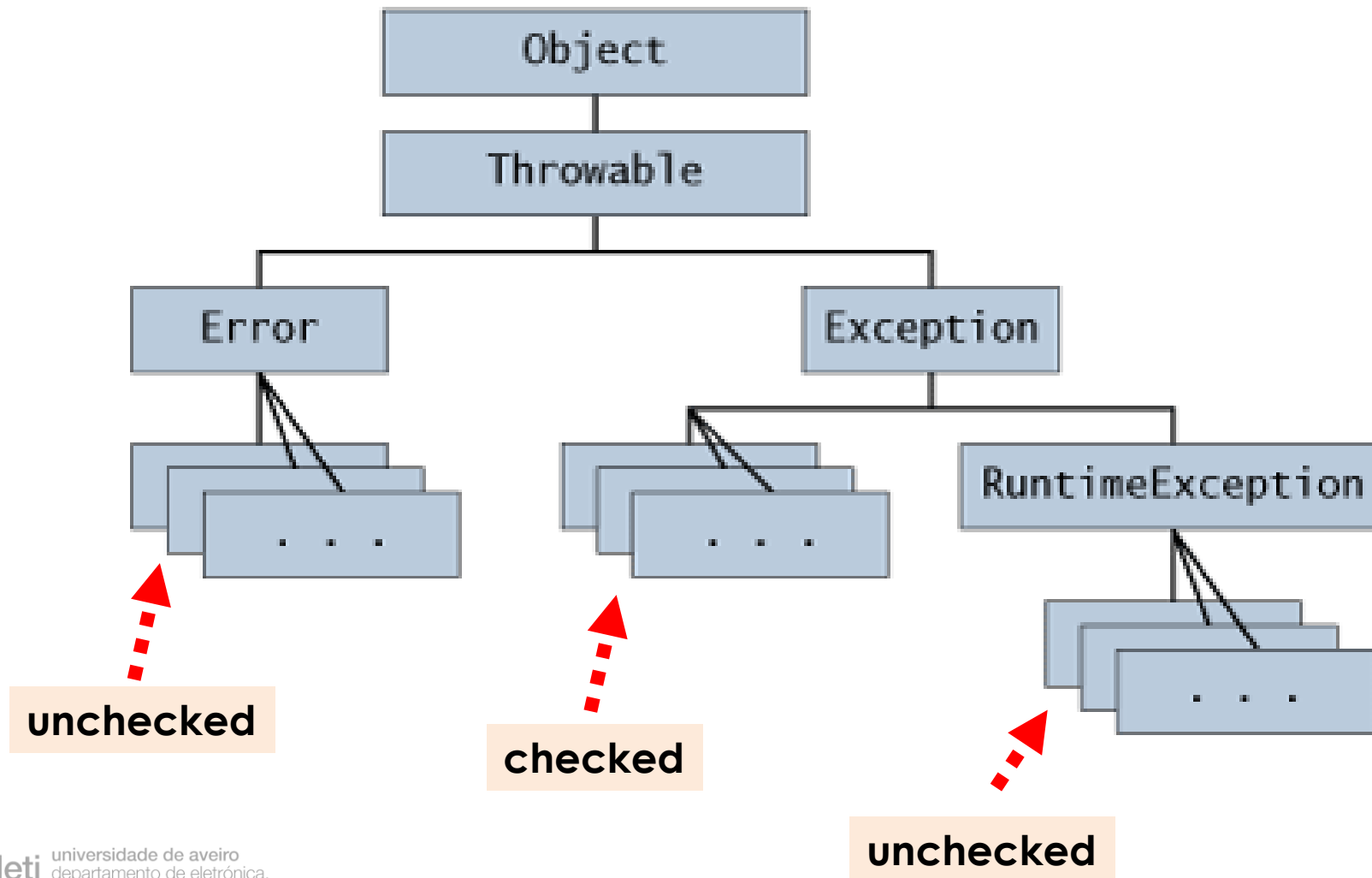
❖ checked

- Se invocarmos um método que gere uma checked exception, **temos de indicar ao compilador como vamos resolvê-la:**
 - 1) Resolver try .. catch, ou
 - 2) Propagar throw (delegar)

❖ unchecked

- **São erros** de programação ou do sistema
- São subclasses de *java.lang.RuntimeException* ou *java.lang.Error*

Exceções - Hierarquia de Classes

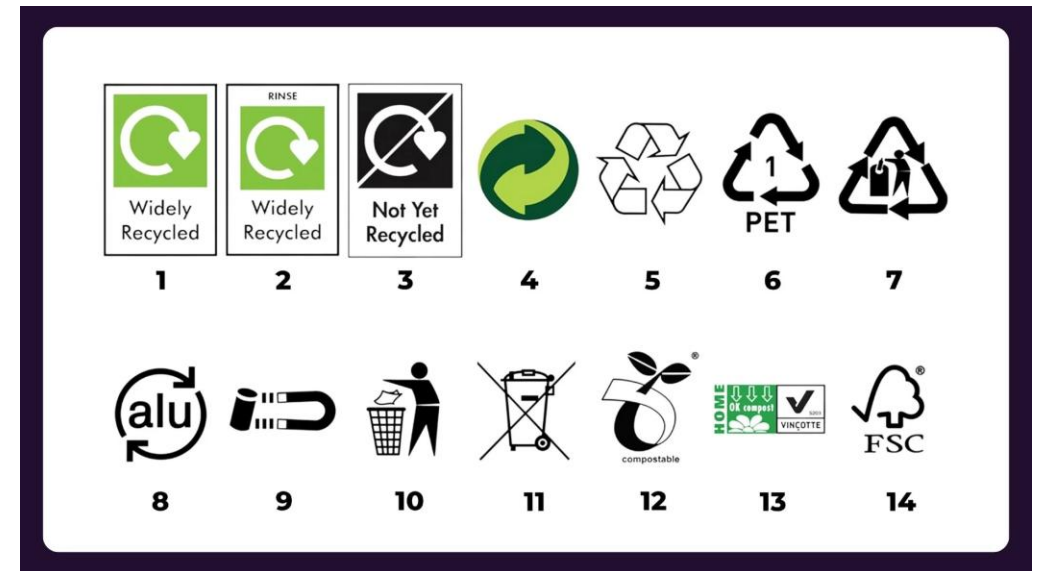


Declaração de Exceções

❖ Quando desenhamos métodos que possam **gerar exceções checked**, temos de assinalá-las explicitamente

❖ Declaração **throws**

```
public void istoPodeDarAsneira()  
    throws TooBigException,  
    TooSmallException,  
    DivByZeroException {  
  
    //...  
  
}
```



Boas Práticas

❖ Usar exceções apenas para condições excepcionais

- Uma API bem desenhada não deve forçar o cliente a usar exceções para controlo de fluxo
- Uma exceção não deve ser usada para um simples teste

X

```
try {  
    s.pop();  
} catch (EmptyStackException es) {...}
```

V

```
if (!s.empty()) s.pop();    // melhor!
```

Boas Práticas

- ❖ Tratar sempre as exceções (ou delegá-las)

```
try {  
    // .. código que pode causar exceções  
} catch (Exception e) {}
```

Boas Práticas

❖ Usar **preferencialmente exceções standards**

- *IllegalArgumentException*
valor de parâmetros inapropriado
- *IllegalStateException*
Estado de objecto incorreto
- *NullPointerException*
- *IndexOutOfBoundsException*

<https://www.baeldung.com/java-common-exceptions>

https://www.tutorialspoint.com/java/java_builtin_exceptions.htm

(most common) Built-in Exceptions in Java

Sr.No.		Exception & Description	
1	ArithmeticException	Arithmetic error, such as divide-by-zero.	
2	ArrayIndexOutOfBoundsException	Array index is out-of-bounds.	
3	ArrayStoreException	Assignment to an array element of an incompatible type.	
4	ClassCastException	Invalid cast.	
5	IllegalArgumentException	Illegal argument used to invoke a method.	
6	IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.	
7	IllegalStateException	Environment or application is in incorrect state.	
8	IllegalThreadStateException	Requested operation not compatible with the current thread state.	
9	IndexOutOfBoundsException	Some type of index is out-of-bounds.	
10	NegativeArraySizeException	Array created with a negative size.	
11	NullPointerException	Invalid use of a null reference.	
12	NumberFormatException	Invalid conversion of a string to a numeric format.	
13	SecurityException	Attempt to violate security.	
14	StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.	
15	UnsupportedOperationException	An unsupported operation was encountered.	
16	ClassNotFoundException	Class not found.	
17	CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.	
18	IllegalAccessException	Access to a class is denied.	
19	InstantiationException	Attempt to create an object of an abstract class or interface.	
20	InterruptedException	One thread has been interrupted by another thread.	
21	NoSuchFieldException	A requested field does not exist.	
22	NoSuchMethodException	A requested method does not exist.	

Sumário

❖ Controlo de Erros

- Bloco try .. catch
- Bloco try-with-resources
- Instrução throw

❖ Exceções

- checked
- unchecked

❖ Declaração throws

The END